

Python

Date:

Page:

script.py → Python को code/script लेखन व चलाउन प्रयोग हुन्छ।

data_analysis.ipynb → Data analysis गर्न Jupyter notebook मा प्रयोग हुन्छ।

program.exe → Computer मा program/application चलाउन प्रयोग हुन्छ।

document.docx → Word document लेखन र edit गर्न प्रयोग हुन्छ।

spreadsheet.xlsx → Excel मा data राख्न, calculation र analysis गर्न प्रयोग हुन्छ।

image.png → Picture/image हेर्न र राख्न प्रयोग हुन्छ।

archive.zip → धेरै files लाई ठुलो compressed file मा राख्न प्रयोग हुन्छ।

styles.css → Website को design, color, font र layout मिलाउन प्रयोग हुन्छ।

```
print("Hello, world!")
```

print () एक function है।

"Hello, world!" Output है जो run करने पर "Hello, world!" आया।

Python Indentation

Example

```
if 5 > 2:
```

```
    print("five is greater than two!")
```

print() को अगली space/tab दे। चूंकि print() लाई if को निम्नले code में Python ले कुछ।

Example:

Syntax Error:

```
if 5 > 2:
```

```
print("five is greater than two!")
```

print() को अगली indentation/space देना। चूंकि Python ले if पाए इस code चलाने में

Structure हुइसन स्वरदन 2 indentation/syntax Error आउछ ।

* Simple rule : if, for, while, def आदि को आखवाक ल्याइको निक्को code indent (Space/tab) दिनुपर्छ ।

Python Variable Naming Rules

- Start character : Must begin with a letter or an underscore (_).
- No leading numbers : Cannot start with a number (eg., 2myvar is illegal).
- Allowed characters : Only letters numbers, and underscores (A-Z, 0-9, _).
- Case sensitivity : Names are case-sensitive (myvar and MYVAR are different).
- No reserved words : cannot use python keywords like if, for, or print.

Page: _____
Date: _____

Breakdown of the Examples in the Image.

- `myvar = "John"` : Valid. Uses all lowercase letters.
- `my_var = "John"` : Valid. Uses an underscore to separate words (Snake Case).
- `_my_var = "John"` : Valid. Starts with an underscore.
- `myVar = "John"` : Valid. Uses a capital letter for the second word (Camel Case).
- `MYVAR = "John"` : Valid. Uses all uppercase letters.
- `myvar2 = "John"` : Valid. Contains a number at the end.

Illegal variable names:

Page:

Page:

* Illegal Examples (for contrast) .

- `2myvar = "John"` : Invalid (starts with a number).
- `my-var = "John"` : Invalid (contains a hyphen).
- `my var = "John"` : Invalid (contains a space).

Many Values to Multiple Variables

Python allows you to assign values to multiple variables in one line:

Example

```
x, y, z = "orange", "Banana", "Apple"  
print(x)  
print(y)  
print(z)
```

Three variables `x`, `y` and `z` are assigned three values at the same time.

∴ then output ;

Orange
Banana
Apple .

} Multiple variables can be assigned multiple values in one line .

1. Python Data structures.

Python has different built-in data types.

- String (str) - text data.
- Integer (int) - whole numbers.
- Float (float) - decimal numbers.
- Complex (complex) - complex numbers.
- List (list) - ordered, changeable collection.
- Tuple (tuple) - ordered, unchangeable collection.
- Range (range) - sequence of numbers.
- Dictionary (dict) - key-value pairs.
- Set (set) - unordered unique values.
- Boolean (bool) - True or False.

2. Type Casting.

Type casting means converting one data type into another.

`int(2.8) #2.`

`int("3") #3.`

`float(2.8) #2.8.`

`float("4.2") #4.2`

3. Python Strings

A string is a sequence of characters written inside quotes.

```
print("Hello World").  
print("It's alright").
```

Multiline string: Use triple quotes.

```
a = """Hello  
World"""  
print(a).
```

4. String Methods

- `Upper()` → converts string to uppercase.
- `lower()` → converts string to lowercase.
- `Strip()` → removes whitespace from beginning and end.
- `replace()` → replaces one string with another.

```
text = "Hello world"  
print(text.upper())  
print(text.lower())  
print(text.strip())  
print(text.replace("H", "J"))
```

Allow Duplicates:Example

List allow duplicate values:

```
thislist = ["apple", "banana", "cherry"]
print(thislist)
```

print(thislist) garda purai print eg:

Output: apple
banana
cherry

[-1] → garda last deki
start hoga.

For eg: `print(thislist[1])` garew vane
Output: banana auka.

Append ItemsExample:

using the `append()` method to append an item.

```
thislist = ["apple", "banana", "cherry"]
thislist.append("orange")
print(thislist)
```

Output apple banana cherry orange

this list.append → gard thapinxa.

" • remove → Eg: remove("orange") garda orange hatxa.

" • insert → add hunx item haru.

" • pop → subtract (-) hunx Eg: ghada.

" • clear → output empty auxa.

Loop Through a list

You can loop through the list items by using a for loop.

Example

print all items by using a for loop:

```
thelist = ["apple", "orange", "Pineapple"]
```

```
for x in thelist:
```

```
    print(x)
```

And put: apple

orange

pineapple

* { print(x) garda
" ([i]) in range ma auxa like same as

Ascending

Date:

Page:

Sort List Alphabetically

List object has a sort() method that will sort the list.

Example

Sort the list alphabetically.

by ascending
by

```
thislist = ["orange", "kiwi", "pineapple", "litchi"]
```

```
thislist.sort()
```

```
print(thislist)
```

Output: kiwi

litchi

orange

pineapple

• sort > print gada

output ascending

order ma auka

= thislist.sort

Descending

```
thislist = ["orange", "Apple", "litchi"]
```

```
thislist.sort(reverse = True)
```

```
print(thislist)
```

Output: orange

litchi

Apple

• sort(reverse = True)

gada descending ma output

auka

Copy a list

Date:

Page: 74

Use the copy() Method

You can use the built-in List method

→ copy() method is used to create a duplicate copy of a list/dictionary.

```
list 1 = [10, 20, 30]
```

```
list 2 = list 1.copy()
```

```
print(list 2)
```

output

[10, 20, 30]

eg: 80

Python has a set of built-in methods that you can use on lists.

Method	Description
--------	-------------

<u>append()</u>	Adds an element at the end of the list.
-----------------	---

<u>clear()</u>	Removes all the elements from the list.
----------------	---

<u>copy()</u>	Returns a copy of the list.
---------------	-----------------------------

<u>count()</u>	Returns the number of elements with the specified value.
----------------	--

`extend()`: Add the elements of a list (or any iterable), to the end of the current list.

`index()`: Returns the index of the first element with the specified value.

`insert()`: Adds an element at the specified position

`pop()`: Removes the element at the specified position.

`remove()`: Removes the item with the specified value.

`reverse()`: Reverses the order of the list

`sort()`: Sorts the list.

Tuple

Date:

Page: 93

1. Tuple :-> Tuple નોંડે python માં multiple values એક variable માં રાખવા માટે data type છે ।

Tuple બનાવવા () round brackets પ્રયોગ કરવામાં આવે છે ।

```
Fruits = ("apple", "banana", "mango")  
print(Fruits)
```

Output :

← ('apple', 'banana', 'mango')

આથી Fruits એક tuple છે ।

2. Tuple Items :-> Tuple માં નોંડેલા પ્રત્યેક value ને item કહેવાય ।

```
Fruits = ("apple", "banana", "mango")
```

↓
આ એક એક item છે ।

તેથી આ 3 વાળા items છે ।

3. Ordered $\Rightarrow$ Ordered tuple में items गुन order में रखेको ह, यही order में रहनेको ।

Fruits = ("apple", "banana", "mango")

यही :

apple : पहिलो

banana : दोस्रो

mango : तेस्रो

apple लाई mango को agadi वा पहिलो आफ्नो change हुदैन।

यसलाई index वाट पनि access गर्न सकिन्छ

Fruits = ("apple", "banana", "mango")

print (Fruits [0])

print (Fruits [1])

print (Fruits [2])

Output : apple

banana

mango

(Tuple ordered हुन्छ)

4) Unchangeable :- Unchangeable means that once a tuple is created, its items cannot be changed, added, or removed.

```
fruits = ("apple", "kiwi", "mango")
fruits[0] = "orange"
```

↑ Syntax error here

→ Syntax chalauda error auka beacuse "apple" lai "orange" mai change nai thiketa.

* List mai change !

```
fruits = ["kiwi", "orange", "Papaya"]
fruits[0] = "apple"
```

∴ Tuple = Unchangeable / Immutable

5) Allow Duplicates :- Duplicate matlab same value dobara baar use kr sakte hai.

Tuple mai duplicate values rakr sktte.

```
fruits = ("apple", "banana", "apple",
          "mango", "banana")
```

```
print(fruits)
```

output :- ('apple', 'banana', 'apple', 'mango', 'banana')

6. Tuple Length $\Rightarrow$ Tuple में कितने वस्तु items हैं, उनके लिये हमें `len()` use करना है।

```
fruits = ("apple", "kiwi", "mango")
print(len(fruits))
```

$\therefore$ Output $\Rightarrow$ 3

क्यों कि हमें tuple में 3 वस्तु items हैं।

$\therefore$ `len()` tuple में कितने वस्तु items हैं, उनके लिये count करेगा।

Term	Meaning
tuple	Multiple values रखने वाला data type.
Tuple Items	Tuple में रखे गए values
ordered	Items का order maintain होगा।
Unchangeable	काइसे भी items change नहीं किया जा सकता।
allow duplicates	एक ही value को बार-बार रख सकते हैं।
Tuple length	<code>len()</code> से total items count करेगा।

$\therefore$ Tuple = ordered + Unchangeable + Allow Duplicates

Create Tuple with one item

Date:

Page:

1. Create Tuple with one item

→ A tuple with one item must have a comma.

```
my_tuple = ("apple",)
```

∴ without comma, it is not a tuple.

2. Tuple Items - Data types

→ Tuple can contain different types.

```
my_tuple = ("Ram", 20, 5.5, True)
```

3. type() → tells us the data type.

```
my_tuple = ("apple", "banana")  
print(type(my_tuple))
```

∴ output → <class 'tuple'>

4. Access Tuple Items

→ use index number to access items.

```
fruits fruits = ("apple", "banana", "mango")  
print(fruits[0])
```

∴ output: apple

5) Dictionary

→ A dictionary stores data in key : value pairs

```
student = {  
    "name" : "Ram",  
    "age" : 20  
}
```

6. Dictionary Items

→ Each key-value pair is called an item.

"name" : "Ram"

Here:

- name → key
- Ram → value

7. Duplicates Not Allowed

→ Dictionaries cannot have two items with same key:

If the same key is written more than one, the latest value replaces the previous value.

```
Student = {
    "name": "Ram",
    "age": 20,
    "age": 25.
```

```
}
print(student)
```

Here, the value of age will be 25.

* Accessing Dictionary Items.

→ we can access a dictionary value by using its key inside square brackets.

```
Car = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
```

```
}
print(car["model"])
```

Output: → Mustang

* change Dictionary values:

→ we can change a value by referring to its key.

```
car["year"] = 2018
```

* Adding Items!

→ we can add a new item by creating a new key and assigning a value.

```
car["color"] = "red"
```

* Removing Items

→ The pop() method removes an item using its key.

```
car.pop("model")
```

* Copying a Dictionary

→ The copy() method creates a copy of a dictionary.

```
mydict = car.copy()
```

* Nested Dictionary :-

→ A dictionary can contain other dictionaries. This is called a nested dictionary.

2. Dictionary Method. Methods

→ Python provides several built-in methods for dictionaries.

Method

Meaning

copy()

Returns a copy of the dictionary.

fromkeys()

Creates a dictionary from specified keys.

get()

Returns the value of a specified key.

items()

Returns key-value pairs

keys()

Returns all keys

pop()

Removes an item using its key

popitem()

Removes the last inserted key-value pair

setdefault()

Return a value and can insert a key if it does not exist

update()

updates the dictionary

values()

Returns all values

3: Python Structural Hierarchy

→ Python programs can be organized into different levels:

* Module

→ A module is a single .py file.

* Package

→ A package is a folder containing multiple modules.

* Library

→ A library is a collection of packages used for a particular purpose.

* Framework

→ A framework provides the overall structure or architecture for building an application.

* Simple order:

Module → Package → Library → Framework

4. Random Module

The random module is used to generate random values or make random choices.

It can be used for programs such as:

- Lucky draw game
- Random people choice game
- Love calculator

for example:-

```
import random
```

```
names = ["saisha", "sanza", "sanvi"]
```

```
winner = random.choice(names)
```

```
print(winner)
```

5. Python Conditions and If statements.

Conditions are used to make decisions in Python.

Comparison Operators

operator	Meaning
==	Equal to
!=	Not Equal to
<	Less than
<=	Less than or equal to
>	Greater than
>=	Greater than or equal to

* IF statement

The if statement executes a block of code when a condition is True.

```
a = 33
```

```
b = 200
```

```
if b > a:  
    print("b is greater than a")
```

Important

Python uses indentation to identify the block of code.

```
if age >= 18:  
    print("You are an adult")
```

6. Multiple statements in IF

We can write statements inside an if block.

```
age = 20
```

```
if age >= 18:  
    print("You are an adult")  
    print("You can vote")  
    print("You have full legal rights")
```

7. Boolean Values in IF

Boolean values are:

- True
- False

A boolean variable can be directly used in an if statement.

is_logged_in = True

```
if is_logged_in:  
    print("welcome back!")
```

8. Elif Statement

elif means "else if".

It is used when we want to check multiple conditions.

score = 75

```
if score >= 90:
    print("Grade : A")
```

```
elif score >= 80:
    print("Grade : B")
```

```
elif score >= 70:
    print("Grade : C")
```

```
else:
    print("Grade : D")
```

∴ Python checks the conditions from top to bottom and executes the first true condition

9. Else Statement

The else statement executes when all previous conditions are false.

a = 200

b = 33

```
if b > a:
```

```
    print("b is greater than a")
```

```
else:
```

```
    print("b is not greater than a")
```

∴ else does not have a condition

10. Logical Operators

Logical operators are used to combine conditions.

Python has 3 logical operators:

and

Returns True only when both conditions are True.

```
if age >= 18 and age <= 60:  
    print("Eligible")
```

or

Returns True when at least one condition is True.

```
if age < 18 or age > 60:  
    print("Not in working age")
```

not

Reverses the result.

```
if not is_logged_in:  
    print("Please login")
```

17. Pass Statement

The pass statement is used when we need a statement syntactically but do not want Python to perform any action.

```
age = 16
```

```
if age < 18:
```

```
    pass
```

```
else:
```

```
    print("Access granted")
```

Uses of Pass

- As a placeholder
- While developing a program
- For empty functions
- For empty classes
- When logic will be added later

12. Nested If statement

An if statement inside another if statement is called a nested if statement.

$x = 41$

```
if x > 10:
```

```
    print("Above ten")
```

```
    if x > 20:
```

```
        print("And also above 20")
```

```
    else:
```

```
        print("But not above 20")
```

13. Practice Programs

The PPT includes practice programs such as:

* BMI Program

A BMI program calculates Body Mass Index based on height and weight.

* Odd or Even

Checks whether a number is odd or even.

```
if number % 2 == 0:  
    print("Even")  
else:  
    print("odd")
```

* Prime or Not

Checks whether a number is a prime number.

* Rock - Paper - Scissors

A game where the user and computer choose:

- Rock
- Paper
- Scissors

* Treasure Island

A decision-making game using multiple if, elif, and else conditions.

14. Python Loops

A loop is used to execute a block of code repeatedly.

* Python has two main types of loops:

1. while loop
2. for loop

15. while loop

A while loop executes a block of code as long as the condition is True.

```
i = 1
```

```
while i < 6:
```

```
    print(i)
```

```
    i += 1
```

output :

1
2
3
4
5

16. Break statement

The break statement is used to stop the loop completely.

```
i = 1
```

```
while i < 6 :
```

```
    print(i)
```

```
    if i == 3:
```

```
        break
```

```
    i += 1
```

∴ When i becomes 3, the loop stops.

Remember :

break = completely stop the loop

17. Continue Statement

The Continue statement skips the current iteration and moves to the next iteration.

```
i = 0
```

```
while i < 6:
```

```
    i += 1
```

```
    if i == 3:
```

```
        continue
```

```
    print(i)
```

Remember:

continue = skip current iteration

18. For Loop

A for loop is used to iterate through a sequence such as a list or string.

```
fruits = ["kiwi", "mango", "orange"]
```

```
for x in fruits:
    print(x)
```

19. Break in for loop

```
fruits = ["kiwi", "mango", "orange"]
```

```
for x in fruits:  
    print(x)
```

```
    if x == "banana":  
        break
```

∴ The loop stops when "~~banana~~ mango" is found.

20. Continue in for loop

```
fruits = ["kiwi", "mango", "orange"]
```

```
for x in fruits:
```

```
    if x == "mango":  
        continue
```

```
    print(x)
```

∴ Here, "Mango" is skipped.

21. Range() Function

The range() function generates a sequence of numbers.

```
for x in range(6):  
    print(x)
```

output : 0

1
2
3
4
5

∴ The ending number is not included.

-X- Start, Stop and Step

```
for x in range(2, 30, 3):  
    print(x)
```

- 2 = starting value
- 30 = ending value
- 3 = step/increment

* Python Function

A function is a block of code that runs when it is called.

Function help us:

- Avoid code repetition
- Organize code
- Reuse code
- Return a result

* creating a function

we use the def keyword.

```
def my_function():  
    print("Hello from a function")
```

* calling a function

```
my_function()
```

∴ The function will execute when we call it.

* Return vs Print.

print()

print() displays a value on the screen.

```
def add(a, b):  
    print(a+b)
```

return

return sends a value back from a function.

```
def add(a, b):  
    return a+b
```

```
result = add(5, 3)  
print(result)
```

- print → shows the result
- return → sends the result back

Calculator Program

A Calculator program can use functions for different operations:

```
def add(a,b):  
    return a+b
```

```
def subtract(a,b)  
    return a-b
```

```
def multiply(a,b)  
    return a*b
```

```
def divide(a,b):  
    return a/b
```

∴ Function make the calculator easier to organize and reuse.